Objetivos generales

- Aplicar los conocimientos adquiridos en el curso para diseñar, implementar y analizar una solución basada en bases de datos NoSQL, utilizando MongoDB y Neo4j para el almacenamiento, procesamiento, consulta y visualización de información.

Objetivos específicos

- Comprender las características, ventajas y casos de uso de MongoDB como base de datos documental y Neo4j como base de datos orientada a grafos.
- Diseñar e implementar una arquitectura que permita integrar una aplicación con MongoDB y Neo4j.
- Realizar procesos de carga, transformación y almacenamiento de datos provenientes de archivos estructurados o semiestructurados.
- Construir consultas analíticas en MongoDB que permitan identificar tendencias, distribuciones y patrones dentro de la información almacenada.
- Modelar nodos, propiedades y relaciones en Neo4j para representar y analizar conexiones entre los diferentes elementos del problema.
- Desarrollar una interfaz gráfica que permita consultar, filtrar y visualizar los resultados obtenidos.
- Analizar las ventajas y limitaciones de utilizar bases de datos documentales y bases de datos de grafos dentro de una misma solución.
- Documentar la arquitectura, configuración, consultas, decisiones técnicas y resultados obtenidos durante el desarrollo del proyecto.

Descripción de la actividad

El presente proyecto tiene como objetivo desarrollar una aplicación para el almacenamiento, procesamiento, análisis y visualización de información relacionada con las evaluaciones de aspirantes a ingresar a la Universidad de San Carlos de Guatemala.

La solución deberá utilizar dos modelos de bases de datos NoSQL: una base de datos documental mediante MongoDB y una base de datos orientada a grafos mediante Neo4j.

MongoDB será utilizado principalmente para almacenar y consultar información relacionada con evaluaciones, aspirantes, carreras, instituciones educativas, ubicaciones y resultados. Debido a su modelo basado en documentos, permitirá representar información semiestructurada y realizar consultas analíticas sobre grandes conjuntos de datos.

Neo4j será utilizado para representar y analizar las relaciones existentes entre los diferentes elementos del sistema, tales como aspirantes, carreras, instituciones educativas, municipios, departamentos, materias y resultados de evaluación.

La aplicación deberá permitir cargar información desde archivos de datos, procesarla, almacenarla en ambos motores de base de datos y presentar los resultados mediante una interfaz gráfica.

El estudiante deberá implementar como mínimo los siguientes componentes:

- Interfaz gráfica o frontend.
- Backend o API.
- Base de datos MongoDB.
- Base de datos Neo4j.
- Proceso de carga y transformación de datos.
- Consultas analíticas.
- Visualización de estadísticas.
- Visualización de relaciones mediante grafos.
- Documentación técnica.

TECNOLOGÍAS

Los sistemas gestores de bases de datos obligatorios para el desarrollo del proyecto serán:

- MongoDB.
- Neo4j.

Las tecnologías utilizadas para el frontend y backend serán de libre elección.

El estudiante podrá seleccionar cualquier lenguaje de programación, framework o biblioteca que considere adecuada para implementar la solución.

Ejemplos de tecnologías permitidas

Frontend

Se podrá utilizar, entre otras tecnologías:

- Angular.
- React.
- Vue.js.
- HTML, CSS y JavaScript.
- Otra tecnología equivalente.

Backend

Se podrá utilizar, entre otras tecnologías:

- Node.js con Express, NestJS u otro framework.
- Java con Spring Boot.
- Python con Flask, FastAPI o Django.
- .NET.
- Otra tecnología equivalente.

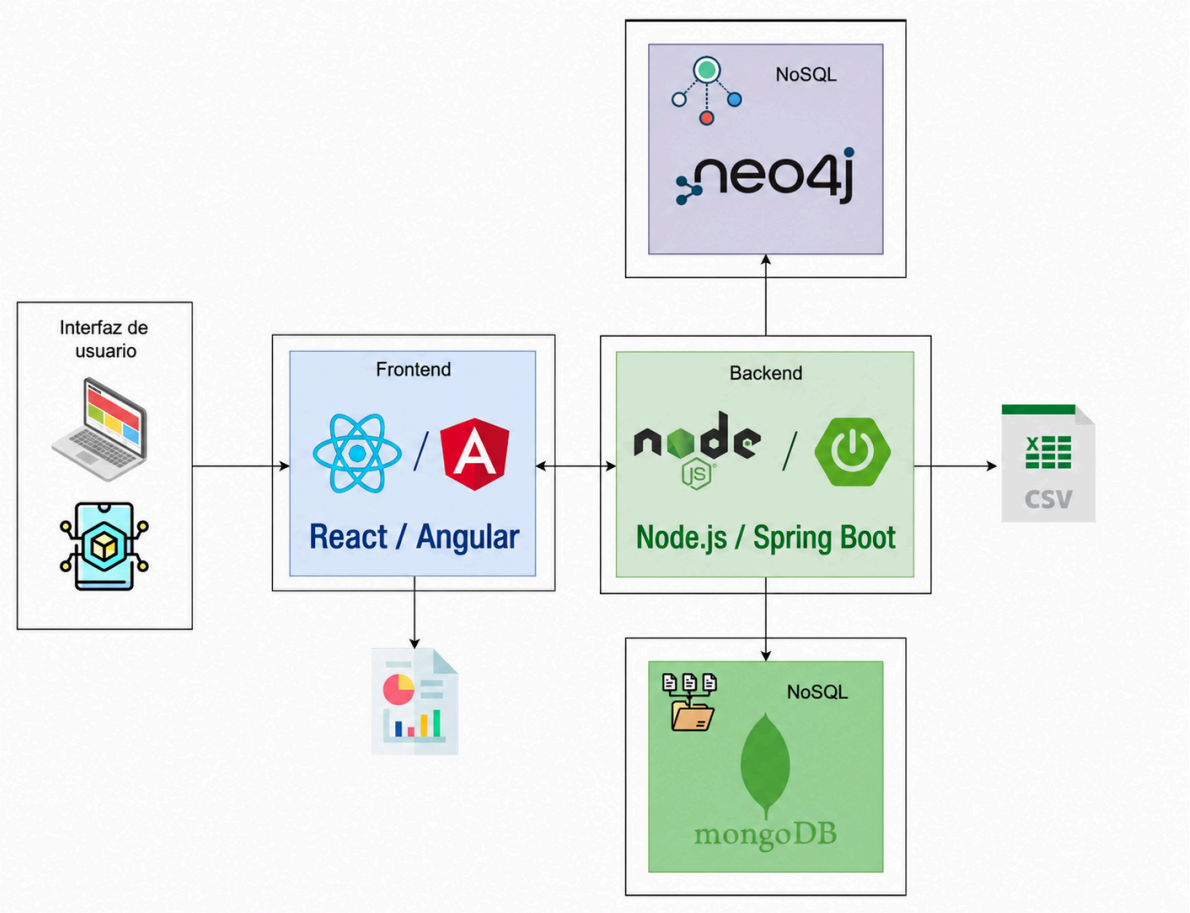
La selección tecnológica deberá ser coherente con la arquitectura propuesta y permitir una correcta integración con MongoDB y Neo4j.

No se calificará el uso de un framework específico. Se evaluará que la solución funcione correctamente, esté estructurada adecuadamente y cumpla con los requerimientos definidos en el proyecto.

CONSIDERACIONES SOBRE LA ARQUITECTURA

La aplicación deberá implementar una separación clara entre la interfaz gráfica, la lógica de aplicación y las bases de datos.

Como referencia, podrá utilizarse la siguiente arquitectura:



La comunicación entre el frontend y las bases de datos deberá realizarse preferiblemente mediante el backend o API.

El frontend no deberá conectarse directamente a MongoDB o Neo4j.

El backend será responsable de:

- Recibir solicitudes desde el frontend.
- Validar los datos recibidos.
- Ejecutar consultas sobre MongoDB.
- Ejecutar consultas sobre Neo4j.
- Procesar los resultados.
- Enviar la información requerida al frontend.
- Manejar errores de forma controlada.

El estudiante podrá implementar una arquitectura diferente siempre que pueda justificar técnicamente su elección y que esta cumpla con los requerimientos establecidos.

PROBLEMA DE APLICACIÓN

La Universidad de San Carlos de Guatemala recibe cada año una gran cantidad de aspirantes interesados en ingresar a las distintas carreras universitarias. Como parte del proceso de admisión, los aspirantes realizan evaluaciones cuyos resultados permiten determinar su rendimiento en diferentes áreas y establecer si cumplen con los requisitos necesarios para continuar con el proceso de ingreso.

Actualmente, la información relacionada con estas evaluaciones puede encontrarse almacenada en archivos estructurados, como archivos CSV, lo que dificulta realizar consultas complejas, identificar patrones, analizar relaciones entre los datos y presentar los resultados de forma clara para la toma de decisiones.

La información contiene datos relacionados con los aspirantes, evaluaciones realizadas, materias, resultados obtenidos, instituciones educativas de procedencia, municipios, departamentos, carreras objetivo y otros elementos relevantes para el análisis académico.

Como profesional en bases de datos NoSQL, se solicita al estudiante diseñar e implementar una aplicación que permita cargar, almacenar, consultar, analizar y visualizar esta información mediante el uso conjunto de MongoDB y Neo4j.

MongoDB deberá utilizarse para almacenar los registros y realizar consultas estadísticas y analíticas sobre la información de los aspirantes y sus evaluaciones.

Neo4j deberá utilizarse para representar las relaciones existentes entre los diferentes elementos del conjunto de datos y permitir consultas orientadas a grafos.

La aplicación deberá ofrecer una interfaz gráfica desde la cual sea posible consultar la información, aplicar filtros, visualizar estadísticas y explorar las relaciones almacenadas en Neo4j.

INFORMACIÓN MÍNIMA A GESTIONAR

Para el desarrollo del proyecto se utilizará el conjunto de datos **Pruebas específicas**, publicado en el portal de Datos Abiertos de la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala.

Este conjunto contiene información relacionada con las pruebas específicas realizadas por aspirantes de primer ingreso a la Facultad de Ingeniería.

La fuente oficial se encuentra disponible en:

Datos Abiertos – Facultad de Ingeniería – Pruebas específicas

https://datosabiertos.ingenieria.usac.edu.gt/en/dataset/pruebas-especificas

El portal proporciona archivos correspondientes a diferentes años, por lo que el estudiante deberá utilizar los archivos de datos indicados para el desarrollo del proyecto.

INFORMACIÓN CONTENIDA EN EL DATASET

Cada registro contiene información relacionada con una evaluación realizada por un aspirante.

Los campos disponibles son los siguientes:

Campo	Descripción
fecha_asignacion	Fecha en la que el aspirante se asignó a la prueba.
sexo	Sexo registrado para el aspirante.
anio_nacimiento	Año de nacimiento del aspirante.
materia	Materia evaluada en la prueba específica.
numero_de_fecha_de_evaluacion	Número de oportunidad o fecha de evaluación correspondiente al año de ingreso.
anio_de_ingreso	Año para el cual el aspirante desea ingresar.
aprobacion	Resultado de la evaluación.
carrera_objetivo	Carrera a la que el aspirante desea ingresar.
departamento_institucion_educativa	Departamento donde se encuentra la institución educativa de procedencia.
municipio_institucion_educativa	Municipio donde se encuentra la institución educativa de procedencia.
tipo_institucion_educativa	Tipo de institución educativa de procedencia.
correlativo_aspirante	Identificador anónimo utilizado para reconocer al aspirante.

Un registro del archivo podrá presentar una estructura similar a la siguiente:


```
{
  "fecha_asignacion": "2014-08-23",
  "sexo": "M",
  "anio_nacimiento": "SIN REGISTRO",
  "materia": "MATEMATICA",
  "numero_de_fecha_de_evaluacion": 1,
  "anio_de_ingreso": 2015,
  "aprobacion": "NO APROBADO",
  "carrera_objetivo": "NINGUNO",
  "departamento_institucion_educativa": "GUATEMALA",
  "municipio_institucion_educativa": "GUATEMALA - CIUDAD CAPITAL ZONA 1",
  "tipo_institucion_educativa": "PRIVADO",
  "correlativo_aspirante": "949d241ac10256edf11339a303c72a50"
}
```


El ejemplo anterior representa los datos tal como pueden encontrarse en el archivo original.

CARGA Y TRANSFORMACIÓN DE DATOS

Los archivos proporcionados no deberán asumirse como información completamente limpia o normalizada.

Como parte del proyecto, el estudiante deberá implementar un proceso de extracción, transformación y carga de datos ETL que permita preparar la información antes de almacenarla o utilizarla dentro del sistema.

El proceso deberá considerar, como mínimo:

- Lectura de los archivos CSV.
- Validación de la estructura de los registros.
- Conversión de tipos de datos cuando corresponda.
- Identificación y tratamiento de valores faltantes o especiales.
- Estandarización de valores cuando sea necesaria.
- Registro de datos válidos e inválidos.
- Carga de la información procesada hacia MongoDB.
- Generación de nodos y relaciones en Neo4j.

El conjunto de datos puede contener valores que requieran tratamiento antes de realizar determinados análisis.

Por ejemplo:

- anio_nacimiento puede contener valores como SIN REGISTRO.
- aprobacion utiliza valores textuales como APROBADO y NO APROBADO.
- carrera_objetivo puede contener valores como NINGUNO.
- Algunos campos pueden presentar diferencias de formato o valores que el estudiante considere necesario estandarizar.

El estudiante deberá definir y documentar las reglas utilizadas para tratar estos casos.

No será obligatorio eliminar todos los registros incompletos. Dependiendo del análisis, el estudiante podrá conservarlos, transformarlos o excluirlos, siempre que la decisión esté técnicamente justificada.

El campo correlativo_aspirante deberá utilizarse como identificador anónimo del aspirante.

Un mismo aspirante puede aparecer en múltiples registros debido a que puede realizar diferentes evaluaciones o intentos.

Por esta razón, cada fila del archivo no debe interpretarse necesariamente como un aspirante diferente.

Esta característica deberá considerarse tanto para el diseño documental en MongoDB como para el diseño del grafo en Neo4j.

PROCESAMIENTO DE LOS ARCHIVOS

El estudiante deberá implementar una funcionalidad que permita cargar uno o varios archivos CSV correspondientes al conjunto de datos proporcionado.

Durante cada proceso de carga se deberá mostrar como mínimo:

- Nombre del archivo.
- Cantidad total de registros leídos.
- Cantidad de registros procesados correctamente.
- Cantidad de registros rechazados o con errores.
- Tiempo utilizado para procesar el archivo.
- Estado final de la operación.

Los registros rechazados deberán poder identificarse y deberá registrarse la causa correspondiente.

FLUJO DE LA APLICACIÓN

La aplicación deberá permitir cargar, procesar, consultar y visualizar la información proveniente de los archivos CSV proporcionados.

El flujo general deberá contemplar las siguientes etapas:

1. Selección y carga de archivos CSV.
2. Validación y transformación de los registros.
3. Almacenamiento de la información en MongoDB.
4. Generación de nodos y relaciones en Neo4j.
5. Consulta de la información almacenada.
6. Visualización de resultados mediante la interfaz gráfica.

Ingreso de datos al sistema

La aplicación deberá permitir realizar la carga masiva de uno o varios archivos CSV desde la interfaz gráfica.

Durante el proceso de carga, el backend deberá encargarse de:

- Recibir el archivo.
- Procesar los registros.
- Validar la información.
- Realizar las transformaciones necesarias.
- Insertar la información correspondiente en MongoDB.
- Generar la información necesaria en Neo4j.
- Reportar el resultado final de la carga.

La implementación del proceso de carga queda a criterio del estudiante.

Interfaz gráfica

La aplicación deberá contar con una interfaz gráfica que permita consultar y visualizar la información procesada.

Como mínimo deberá incluir los siguientes componentes:

Dashboard de estadísticas

Deberá presentar información relevante obtenida mediante las consultas realizadas en MongoDB.

El estudiante podrá utilizar gráficas de barras, pastel, líneas, indicadores, tablas u otros elementos visuales que considere adecuados.

La biblioteca utilizada para la generación de gráficas será de libre elección.

Explorador de grafos

La aplicación deberá incluir una sección que permita visualizar gráficamente información obtenida desde Neo4j.

El estudiante deberá definir los filtros y criterios de visualización que considere necesarios para facilitar la exploración de los datos.

Buscador

La aplicación deberá permitir realizar búsquedas sobre la información almacenada.

El buscador podrá considerar criterios como:

- Aspirante.
- Carrera objetivo.
- Institución educativa.
- Materia.
- Municipio.
- Departamento.

Se podrán incorporar criterios adicionales.

Manejo de errores

La aplicación deberá manejar adecuadamente los errores que puedan producirse durante:

- La carga de archivos.
- El procesamiento de registros.
- La conexión con MongoDB.
- La conexión con Neo4j.
- La ejecución de consultas.
- La visualización de resultados.

Los errores deberán mostrarse de forma clara y no deberán provocar que la aplicación finalice inesperadamente.

CONSULTAS SOLICITADAS

La aplicación deberá implementar y visualizar los resultados de las siguientes consultas utilizando MongoDB.

El estudiante deberá decidir la forma adecuada de resolver cada consulta según el modelo documental implementado.

MongoDB

1. **Aspirantes por tipo de institución educativa**
   Mostrar la cantidad de aspirantes agrupados por tipo de institución educativa.
2. **Cantidad de aprobados por materia**
   Mostrar la cantidad de evaluaciones aprobadas agrupadas por materia.
3. **Aprobados por carrera y año de ingreso**
   Mostrar la cantidad de aspirantes aprobados agrupados por carrera objetivo y año de ingreso.
4. **Porcentaje de aprobación por materia**
   Calcular el porcentaje de aprobación correspondiente a cada materia.
5. **Promedio de edad por carrera**
   Calcular la edad promedio de los aspirantes agrupados por carrera objetivo.
6. **Creación de colección auxiliar resumen_carrera**
   Generar una colección que almacene información resumida por carrera utilizando los datos procesados.
7. **Promedio de edad de aspirantes aprobados por carrera y tipo de institución**
   Mostrar la edad promedio de los aspirantes que aprobaron, agrupada por carrera objetivo y tipo de institución educativa.
8. **Distribución de aprobados por municipio y carrera objetivo**
   Mostrar la cantidad de aprobados según el municipio de la institución educativa y la carrera objetivo.
9. **Cantidad de evaluaciones por mes y materia para instituciones públicas**
   Mostrar la cantidad de evaluaciones realizadas por mes y materia, considerando únicamente aspirantes provenientes de instituciones públicas.
10. **Top 5 de carreras más demandadas por aspirantes entre 16 y 18 años**
    Determinar las cinco carreras objetivo con mayor cantidad de aspirantes dentro de ese rango de edad.
11. **Historial de desempeño por aspirante**
    Consultar el historial de un aspirante mostrando sus evaluaciones, materias, intentos y resultados.
12. **Distribución por sexo y tipo de institución educativa**
    Mostrar la cantidad de aspirantes agrupados por sexo y tipo de institución.
13. **Tasa de aprobación por edad**
    Calcular el porcentaje de aprobación correspondiente a cada edad.
14. **Número promedio de intentos por materia**
    Determinar el promedio de oportunidades de evaluación utilizadas por los aspirantes para cada materia.
15. **Historial completo de un aspirante**
    Mostrar toda la información disponible de un aspirante identificado mediante su correlativo_aspirante.
16. **Carreras con más aspirantes reprobados en primer intento**
    Determinar las carreras objetivo con mayor cantidad de aspirantes que no aprobaron en su primera oportunidad de evaluación.
17. **Top 5 municipios con mayor cantidad de aspirantes**
    Determinar los cinco municipios de procedencia de instituciones educativas con mayor cantidad de aspirantes registrados.

18. **Tasa de aprobación por tipo de institución educativa**
    Calcular y comparar el porcentaje de aprobación entre los diferentes tipos de institución educativa.
19. **Carreras más demandadas por departamento**
    Determinar cuáles son las carreras objetivo con mayor demanda dentro de cada departamento de procedencia de la institución educativa.
20. **Evolución de aprobación por año de ingreso**
    Mostrar la cantidad y porcentaje de evaluaciones aprobadas y no aprobadas agrupadas por año de ingreso.

PRESENTACIÓN DE RESULTADOS

Los resultados de las consultas deberán visualizarse dentro de la aplicación.

Dependiendo de la naturaleza de cada consulta, podrán utilizarse:

- Gráficas.
- Tablas.
- Indicadores.
- Tarjetas estadísticas.
- Listados.
- Otros componentes visuales que el estudiante considere adecuados.

No será obligatorio utilizar el mismo tipo de visualización para todas las consultas.

El estudiante deberá seleccionar una representación que permita interpretar adecuadamente los resultados obtenidos.

NEO4J

Los datos proporcionados deberán cargarse también en una instancia de Neo4j.

El estudiante deberá diseñar el modelo de grafos identificando los nodos, propiedades y relaciones que considere adecuados para representar la información del conjunto de datos.

Como parte del proyecto se deberá:

1. Cargar la información necesaria en Neo4j.
2. Evitar la creación innecesaria de nodos duplicados.
3. Implementar relaciones que permitan representar conexiones relevantes entre los datos.
4. Integrar las consultas de Neo4j con el backend de la aplicación.
5. Visualizar al menos un grafo desde la interfaz gráfica.
6. Permitir aplicar al menos un filtro sobre la visualización del grafo.

CONSULTAS DE GRAFOS

El estudiante deberá implementar como mínimo **cinco consultas en Neo4j**, definidas por el propio estudiante.

Las consultas deberán demostrar el uso de:

- Nodos.
- Relaciones.
- Propiedades.
- Filtros.
- Recorridos sobre el grafo.

Las cinco consultas deberán ser diferentes entre sí y deberán proporcionar información útil relacionada con el problema planteado.

El estudiante deberá documentar cada consulta Cypher utilizada y explicar brevemente qué información permite obtener.

Durante la calificación se podrá solicitar la modificación o creación de una consulta adicional sobre el modelo implementado.

Importante

- El proyecto se realizará de forma individual.
- Los sistemas gestores de bases de datos obligatorios serán **MongoDB y Neo4j**.
- Las tecnologías utilizadas para frontend y backend serán de libre elección.
- Se permite utilizar software libre para el desarrollo del proyecto.
- Es válido utilizar cualquier IDE, editor de texto, cliente de base de datos o herramienta de desarrollo que el estudiante considere necesaria.
- El estudiante deberá desarrollar y configurar su propia solución, incluyendo frontend, backend, conexiones con las bases de datos y proceso de carga de información.
- El estudiante deberá implementar el proceso de carga, transformación y validación de los archivos CSV proporcionados.
- El modelo documental utilizado en MongoDB y el modelo de grafos utilizado en Neo4j deberán ser definidos por el estudiante.
- Todas las consultas solicitadas deberán poder ejecutarse y visualizarse desde la aplicación.
- El estudiante deberá documentar las consultas realizadas en MongoDB y Neo4j.
- Se deberá utilizar Git y GitHub para el control de versiones del proyecto.

- El repositorio deberá contener como mínimo **5 commits significativos** realizados durante el desarrollo.
- No se permite copiar total o parcialmente el trabajo de otro estudiante.
- No se permite copiar soluciones completas de Internet, repositorios públicos, herramientas de inteligencia artificial u otras fuentes sin comprender, revisar, adaptar y justificar su contenido.
- El uso de herramientas de inteligencia artificial está permitido como apoyo, siempre que el estudiante comprenda, revise y valide el contenido generado.
- El uso inadecuado de inteligencia artificial estará sujeto a una **penalización del 70 % de la nota obtenida**.
- Se considerará uso inadecuado cuando el estudiante no pueda explicar, modificar o reproducir código, consultas, configuraciones o procedimientos incluidos en su proyecto.
- Las copias totales o parciales del proyecto obtendrán una calificación de cero y serán notificadas a la coordinación correspondiente.
- Durante la calificación se podrá solicitar al estudiante realizar modificaciones al código, ejecutar consultas adicionales o explicar cualquier componente de la solución.
- Durante la calificación podrá solicitarse la creación o modificación de una consulta en MongoDB o Neo4j.
- El estudiante deberá obtener como mínimo el **5 % de la calificación total del proyecto** para tener derecho a las siguientes actividades o proyectos del curso.
- Las entregas fuera de la fecha establecida estarán sujetas a las políticas de penalización del curso.

Entrega

El proyecto deberá almacenarse en un repositorio de GitHub.

El repositorio deberá contener como mínimo:

- Código fuente del frontend.
- Código fuente del backend.
- Scripts utilizados para la carga y transformación de los datos.
- Consultas de MongoDB.
- Consultas Cypher de Neo4j.
- Análisis de resultados
- Archivos de configuración necesarios para ejecutar el proyecto.
- Conclusiones
- Manual técnico.
- Manual de usuario.

La fecha de entrega es el día viernes **30 de octubre de 2026**.

Calificación

Pendiente de definir.